Adding Real-Time Chat to SocialNest

Django Channels + WebSockets: frontend to backend, step by step

A task-by-task implementation guide with project structure, database schema, complete code, and a ready-to-paste AI prompt for every task.

Target project	codeAlpha_SocialNest (Django social media app)
Reference video	The YouTube playlist you shared (PL5E1F5cTSTtRSP3Qb8-gZ-Hm5AXp3VKvu). I could not open it, so this guide does not summarise it.
Stack	Django 4.2+, Channels 4, Daphne (ASGI), channels-redis (production), vanilla JS WebSocket API
Features	1-to-1 private chat, message history, unread badges, typing indicator, auto-reconnect, authentication and membership checks
Tasks	15 small tasks, each with goal, steps, code, a done-check and an AI prompt

Read this first (assumptions)

I could not open the GitHub repository (robots.txt blocks automated access) or the YouTube page (blocked), so this guide is **not** copied from either one. It follows the standard, current Django Channels chat pattern that such a video series teaches, applied to a typical CodeAlpha SocialNest layout: Django project with apps for accounts/profiles and posts, server-rendered templates, and Django's built-in User model.

- Project package is called `socialnest`. If yours differs (check the folder that contains settings.py), replace it everywhere.
- Base template is `templates/base.html` with a `{% block content %}`. Adjust block names if yours differ.
- Your profile page URL has the target user's id or username available, so you can add a Message button.
- If your repo differs, paste your real structure into the prompts below and the AI will adapt the code.

Part 1. How Real-Time Chat Works

Normal Django is request/response over HTTP: the browser asks, the server answers, the connection closes. Chat needs the server to push messages to the browser the moment they arrive. WebSockets keep one connection open in both directions. Django alone cannot do that, so we add **Channels**, which lets Django speak ASGI (the async successor to WSGI).

Message flow (User A sends a message to User B)

```
  Browser A (JS)                    Django / Daphne (ASGI)                        Browser B (JS)
  --------------                    ----------------------                        --------------
  new WebSocket("/ws/chat/7/") -->  AuthMiddlewareStack (reads session cookie)
                                    URLRouter -> ChatConsumer.connect()
                                       - is user logged in?  is user in room 7?
                                       - group_add("chat_7", channel_name)
  send({"message":"hi"})       -->  ChatConsumer.receive()
                                       - validate + save Message row in database
                                       - group_send("chat_7", {...})  --> Channel layer (memory / Redis)
                                                                                  |
                                    ChatConsumer.chat_message()  <----------------+  (runs for A's and B's consumers)
  onmessage -> render bubble   <--  self.send(...)                  ----------------------------> onmessage -> render
```

Piece	Role
Daphne	ASGI server. Serves normal HTTP and WebSocket connections on the same port.
asgi.py	Entry point. Routes HTTP to Django and WebSocket to Channels (ProtocolTypeRouter).
AuthMiddlewareStack	Puts the logged-in user into `scope['user']` using the session cookie.
routing.py	WebSocket equivalent of urls.py. Maps `ws/chat/<id>/` to a consumer.
Consumer	WebSocket equivalent of a view. Handles connect, receive, disconnect.
Channel layer	Message bus so different consumers (different users, even different processes) can talk. InMemory for dev, Redis for production.
Group	A named set of connections (one per chat room). group_send delivers to everyone in it.

Part 2. Project Structure

Before (typical SocialNest)

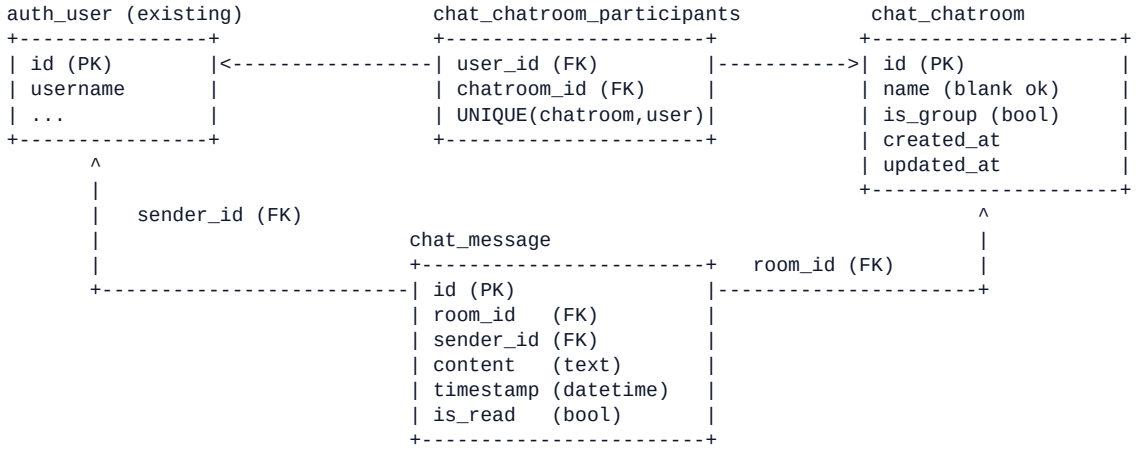
```
codeAlpha_SocialNest/
|-- manage.py
|-- requirements.txt
|-- socialnest/                # project package (settings, urls, wsgi/asgi)
|   |-- settings.py
|   |-- urls.py
|   |-- asgi.py
|   `-- wsgi.py
|-- accounts/                  # users, profiles, follow
|-- posts/                     # posts, likes, comments
|-- templates/
|   |-- base.html
|   `-- ...
`-- static/
```

After (new and changed files marked)

```
codeAlpha_SocialNest/
|-- manage.py
|-- requirements.txt                  [CHANGED]  + channels, daphne, channels-redis
|-- socialnest/
|   |-- settings.py                   [CHANGED]  daphne, channels, ASGI_APPLICATION, CHANNEL_LAYERS
|   |-- urls.py                       [CHANGED]  include("chat.urls")
|   `-- asgi.py                       [CHANGED]  ProtocolTypeRouter (http + websocket)
|-- chat/                             [NEW APP]
|   |-- __init__.py
|   |-- admin.py                      [NEW]  register ChatRoom, Message
|   |-- apps.py
|   |-- consumers.py                  [NEW]  ChatConsumer (WebSocket logic)
|   |-- routing.py                    [NEW]  websocket_urlpatterns
|   |-- models.py                     [NEW]  ChatRoom, Message
|   |-- views.py                      [NEW]  inbox, start_chat, room, history
|   |-- urls.py                       [NEW]  HTTP routes
|   |-- context_processors.py         [NEW]  unread count for navbar (optional)
|   |-- tests.py                      [NEW]
|   `-- migrations/
|-- templates/
|   |-- base.html                     [CHANGED]  Messages link + badge
|   |-- accounts/profile.html         [CHANGED]  "Message" button
|   `-- chat/
|       |-- inbox.html                [NEW]
|       `-- room.html                 [NEW]
`-- static/chat/
    |-- chat.css                      [NEW]
    `-- chat.js                       [NEW]
```

Part 3. Database Schema

Two new tables are enough for 1-to-1 and group chat. A third (auto-created) join table links users to rooms. Existing user and profile tables are untouched.



chat_chatroom

Column	Type	Notes
id	BigAutoField, PK	Used in the WebSocket URL: ws/chat/<id>/
name	CharField(100), blank	Only used for group chats
is_group	Boolean, default False	False = direct 1-to-1 chat
created_at	DateTime, auto_now_add	
updated_at	DateTime	Bumped on every new message; inbox sorts by this

chat_chatroom_participants (auto-created by ManyToManyField)

Column	Type	Notes
chatroom_id	FK to chat_chatroom	ON DELETE CASCADE
user_id	FK to auth_user	ON DELETE CASCADE; unique together with chatroom_id

chat_message

Column	Type	Notes
id	BigAutoField, PK	
room_id	FK to chat_chatroom	related_name='messages'; CASCADE
sender_id	FK to auth_user	related_name='sent_messages'; CASCADE
content	TextField	Validated server-side: not empty, max 2000 chars
timestamp	DateTime, auto_now_add, indexed	Composite index (room_id, timestamp) for fast history
is_read	Boolean, default False	Set True when the recipient opens the room

Design decisions

- **One room per pair:** a helper looks for an existing non-group room containing both users before creating a new one, so A-to-B and B-to-A share the same history.

- **Persist first, broadcast second:** the consumer saves the message before sending it to the group, so nothing shown on screen is ever missing from the database.
- **Rooms, not user pairs:** the same schema supports group chat later with no migration.
- **Possible extensions:** MessageAttachment(message FK, file), ReadReceipt(message FK, user FK, read_at) for group read status, UserPresence(user, last_seen).

Part 4. Implementation Tasks

Do the tasks in order. Each one is small enough to finish and verify before moving on. Commit to git after every task so you can roll back. Every task ends with a **Done when** check and an AI prompt you can paste if you want an assistant to do or review the work.

TASK 1 Install dependencies and create a branch ~5 min

Goal: Add Channels, Daphne and the Redis layer to the project.

terminal

```
git checkout -b feature/chat
pip install "channels[daphne]" channels-redis
pip freeze | findstr /i "channels daphne"        # Windows   (use grep -i on Mac/Linux)
```

Add the same packages to `requirements.txt` (for example `channels[daphne]>=4.0`, `channels-redis>=4.1`).

Done when: `python -c "import channels, daphne; print(channels.__version__)"` prints a version.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> I have a Django project called socialnest (apps: accounts, posts). I want to add real-time chat with Django Channels 4 and Daphne. Give me the exact pip install command, the requirements.txt lines, and tell me which Django versions are compatible. Do not write any other code yet.

TASK 2 Configure settings.py for ASGI and the channel layer ~10 min

Goal: Make Django run through Daphne and give Channels a message bus.

socialnest/settings.py

```
INSTALLED_APPS = [
    "daphne",                    # MUST be first so runserver uses ASGI
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "channels",
    # ... your existing apps (accounts, posts) ...
    "chat",
]

ASGI_APPLICATION = "socialnest.asgi.application"

# Development: in-memory layer (single process only)
CHANNEL_LAYERS = {
    "default": {"BACKEND": "channels.layers.InMemoryChannelLayer"},
}

# Production: switch to Redis (see Part 6)
# CHANNEL_LAYERS = {
#     "default": {
#         "BACKEND": "channels_redis.core.RedisChannelLayer",
#         "CONFIG": {"hosts": [("127.0.0.1", 6379)]},
#     },
# }
```

```
LOGIN_URL = "/accounts/login/"      # adjust to your login route
```

Done when: `python manage.py runserver` prints 'Starting ASGI/Daphne ... development server'.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Here is my settings.py: <paste file>. Add Django Channels configuration: put 'daphne' first in INSTALLED_APPS, add 'channels' and a new 'chat' app, set ASGI_APPLICATION for the package socialnest, and add CHANNEL_LAYERS using InMemoryChannelLayer for dev with a commented Redis alternative. Return only the changed blocks, not the whole file.

TASK 3 Create the chat app ~3 min

Goal: Generate the app skeleton and the folders for templates and static files.

terminal

```
python manage.py startapp chat
mkdir templates\chat static\chat          # Mac/Linux: mkdir -p templates/chat static/chat
```

Done when: A `chat/` folder exists and 'chat' is in INSTALLED_APPS.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Show me the commands to create a Django app named chat inside my existing project, create templates/chat and static/chat folders, and list the files I should add inside chat/ (consumers.py, routing.py, urls.py, context_processors.py).

TASK 4 Write the database models and migrate ~15 min

Goal: Create ChatRoom and Message exactly as in the schema.

chat/models.py

```
from django.conf import settings
from django.db import models


class ChatRoom(models.Model):
    name = models.CharField(max_length=100, blank=True)
    is_group = models.BooleanField(default=False)
    participants = models.ManyToManyField(
        settings.AUTH_USER_MODEL, related_name="chat_rooms"
    )
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-updated_at"]

    def __str__(self):
        return self.name or f"Room {self.pk}"

    def other_user(self, user):
        """For a direct chat: return the person who is not `user`."""
        return self.participants.exclude(pk=user.pk).first()

    @classmethod
    def get_or_create_direct(cls, user_a, user_b):
        room = (
            cls.objects.filter(is_group=False, participants=user_a)
            .filter(participants=user_b)
            .first()
        )
        if room:
            return room, False
        room = cls.objects.create(is_group=False)
        room.participants.add(user_a, user_b)
```

```
        return room, True


class Message(models.Model):
    room = models.ForeignKey(ChatRoom, related_name="messages", on_delete=models.CASCADE)
    sender = models.ForeignKey(
        settings.AUTH_USER_MODEL, related_name="sent_messages", on_delete=models.CASCADE
    )
    content = models.TextField()
    timestamp = models.DateTimeField(auto_now_add=True, db_index=True)
    is_read = models.BooleanField(default=False)

    class Meta:
        ordering = ["timestamp"]
        indexes = [models.Index(fields=["room", "timestamp"])]

    def __str__(self):
        return f"{self.sender}: {self.content[:30]}"
```

chat/admin.py

```
from django.contrib import admin
from .models import ChatRoom, Message

@admin.register(ChatRoom)
class ChatRoomAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "is_group", "updated_at")
    filter_horizontal = ("participants",)

@admin.register(Message)
class MessageAdmin(admin.ModelAdmin):
    list_display = ("id", "room", "sender", "short", "timestamp", "is_read")
    list_filter = ("is_read",)

    def short(self, obj):
        return obj.content[:40]
```

terminal

```
python manage.py makemigrations chat
python manage.py migrate
```

Done when: Both tables appear in the admin at /admin/ and you can create a ChatRoom with two participants.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Create Django models for a chat app: ChatRoom (name blank allowed, is_group bool, participants M2M to AUTH_USER_MODEL, created_at, updated_at) with a classmethod get_or_create_direct(user_a, user_b) that returns an existing non-group room containing both users or creates one, and an other_user(user) method. Message (room FK related_name messages, sender FK, content TextField, timestamp auto_now_add indexed, is_read bool) with a composite index on (room, timestamp). Also write admin.py registrations.

TASK 5 Write the WebSocket consumer ~25 min

Goal: The heart of the feature: authenticate, authorise, save, and broadcast.

chat/consumers.py

```
import json

from channels.db import database_sync_to_async
from channels.generic.websocket import AsyncWebsocketConsumer
from django.utils import timezone

from .models import ChatRoom, Message

MAX_LEN = 2000
```

```
class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.user = self.scope["user"]
        self.room_id = int(self.scope["url_route"]["kwargs"]["room_id"])
        self.group_name = f"chat_{self.room_id}"

        # 1) must be logged in   2) must belong to this room
        if not self.user.is_authenticated or not await self.is_member():
            await self.close(code=4403)
            return

        await self.channel_layer.group_add(self.group_name, self.channel_name)
        await self.accept()

    async def disconnect(self, code):
        if hasattr(self, "group_name"):
            await self.channel_layer.group_discard(self.group_name, self.channel_name)

    # ---- browser -> server ----------------------------------------------
    async def receive(self, text_data=None, bytes_data=None):
        try:
            data = json.loads(text_data or "{}")
        except json.JSONDecodeError:
            return
        kind = data.get("type", "message")

        if kind == "message":
            content = (data.get("message") or "").strip()
            if not content or len(content) > MAX_LEN:
                return
            saved = await self.save_message(content)
            await self.channel_layer.group_send(
                self.group_name, {"type": "chat.message", **saved}
            )

        elif kind == "typing":
            await self.channel_layer.group_send(
                self.group_name,
                {
                    "type": "chat.typing",
                    "sender_id": self.user.id,
                    "is_typing": bool(data.get("is_typing")),
                },
            )

        elif kind == "read":
            await self.mark_read()
            await self.channel_layer.group_send(
                self.group_name, {"type": "chat.read", "reader_id": self.user.id}
            )

    # ---- server -> browser (one handler per event "type") ----------------
    async def chat_message(self, event):
        await self.send(text_data=json.dumps({
            "type": "message",
            "id": event["id"],
            "message": event["message"],
            "sender": event["sender"],
            "sender_id": event["sender_id"],
            "timestamp": event["timestamp"],
        }))

    async def chat_typing(self, event):
        if event["sender_id"] != self.user.id:          # do not echo to the typist
            await self.send(text_data=json.dumps({
                "type": "typing",
                "sender_id": event["sender_id"],
                "is_typing": event["is_typing"],
            }))

    async def chat_read(self, event):
```

```
        if event["reader_id"] != self.user.id:
            await self.send(text_data=json.dumps({"type": "read"}))

    # ---- database helpers (sync ORM wrapped for async) ---------------------
    @database_sync_to_async
    def is_member(self):
        return ChatRoom.objects.filter(pk=self.room_id, participants=self.user).exists()

    @database_sync_to_async
    def save_message(self, content):
        msg = Message.objects.create(room_id=self.room_id, sender=self.user, content=content)
        ChatRoom.objects.filter(pk=self.room_id).update(updated_at=timezone.now())
        return {
            "id": msg.id,
            "message": msg.content,
            "sender": self.user.username,
            "sender_id": self.user.id,
            "timestamp": msg.timestamp.isoformat(),
        }

    @database_sync_to_async
    def mark_read(self):
        Message.objects.filter(room_id=self.room_id, is_read=False).exclude(
            sender=self.user
        ).update(is_read=True)
```

Key rules to remember

- Event `"type": "chat.message"` is delivered to the method `chat_message` (dots become underscores).
- Never call the ORM directly inside async code. Wrap it with `database_sync_to_async`.
- Close with a code (4403) when unauthorised so the browser can tell why it was rejected.
- Always validate length and content on the server. Never trust the browser.

Done when: The file imports without errors: `python manage.py check`.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Write a Django Channels AsyncWebsocketConsumer called ChatConsumer for a 1-to-1 chat. Requirements: room_id comes from scope url_route; reject (close code 4403) unauthenticated users and users who are not participants of the ChatRoom; join group chat_<room_id>; handle JSON messages of type 'message' (strip, reject empty or over 2000 chars, save a Message with database_sync_to_async, bump room.updated_at, group_send), 'typing' (do not echo to sender) and 'read' (mark other user's messages is_read=True and notify). Use handler names chat_message, chat_typing, chat_read. Models are in chat/models.py.

TASK 6 WebSocket routing and asgi.py ~10 min

Goal: Tell Django how to send WebSocket traffic to the consumer.

chat/routing.py

```
from django.urls import re_path
from . import consumers

websocket_urlpatterns = [
    re_path(r"^ws/chat/(?P<room_id>\d+)/$", consumers.ChatConsumer.as_asgi()),
]
```

socialnest/asgi.py (replace the file)

```
import os

from django.core.asgi import get_asgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "socialnest.settings")
```

```
# Must run BEFORE importing anything that touches models
django_asgi_app = get_asgi_application()

from channels.auth import AuthMiddlewareStack                    # noqa: E402
from channels.routing import ProtocolTypeRouter, URLRouter       # noqa: E402
from channels.security.websocket import AllowedHostsOriginValidator  # noqa: E402

import chat.routing                                              # noqa: E402

application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AllowedHostsOriginValidator(
        AuthMiddlewareStack(URLRouter(chat.routing.websocket_urlpatterns))
    ),
})
```

Why AllowedHostsOriginValidator? It blocks cross-site WebSocket hijacking by rejecting connections whose Origin header is not in ALLOWED_HOSTS. Keep it.

Done when: `python manage.py runserver` starts with no import errors.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Create chat/routing.py with a websocket_urlpatterns entry for ws/chat/<int room_id>/ pointing to ChatConsumer.as_asgi(), and rewrite socialnest/asgi.py using ProtocolTypeRouter with http handled by get_asgi_application() and websocket wrapped in AllowedHostsOriginValidator and AuthMiddlewareStack. Make sure get_asgi_application() is called before importing chat.routing.

TASK 7 HTTP views and URLs ~20 min

Goal: Pages for the inbox, starting a chat from a profile, and the room itself, plus a history endpoint.

chat/views.py

```
from django.contrib.auth import get_user_model
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse
from django.shortcuts import get_object_or_404, redirect, render

from .models import ChatRoom, Message

User = get_user_model()
PAGE_SIZE = 30


@login_required
def inbox(request):
    rooms = request.user.chat_rooms.all()
    conversations = []
    for room in rooms:
        conversations.append({
            "room": room,
            "other": room.other_user(request.user),
            "last": room.messages.order_by("-timestamp").first(),
            "unread": room.messages.filter(is_read=False).exclude(sender=request.user).count(),
        })
    return render(request, "chat/inbox.html", {"conversations": conversations})


@login_required
def start_chat(request, user_id):
    other = get_object_or_404(User, pk=user_id)
    if other == request.user:
        return redirect("chat:inbox")
    room, _ = ChatRoom.get_or_create_direct(request.user, other)
    return redirect("chat:room", room_id=room.id)
```

```
@login_required
def room(request, room_id):
    room = get_object_or_404(ChatRoom, pk=room_id, participants=request.user)   # 404 if not a member
    latest = list(room.messages.select_related("sender").order_by("-timestamp")[:PAGE_SIZE])
    latest.reverse()
    room.messages.filter(is_read=False).exclude(sender=request.user).update(is_read=True)
    return render(request, "chat/room.html", {
        "room": room,
        "other": room.other_user(request.user),
        "chat_messages": latest,
    })


@login_required
def history(request, room_id):
    """JSON: older messages for infinite scroll. Usage: ?before=<message id>"""
    room = get_object_or_404(ChatRoom, pk=room_id, participants=request.user)
    qs = room.messages.select_related("sender").order_by("-id")
    before = request.GET.get("before")
    if before and before.isdigit():
        qs = qs.filter(id__lt=int(before))
    items = list(qs[:PAGE_SIZE])
    items.reverse()
    return JsonResponse({
        "messages": [{
            "id": m.id, "message": m.content, "sender": m.sender.username,
            "sender_id": m.sender_id, "timestamp": m.timestamp.isoformat(),
        } for m in items],
        "has_more": len(items) == PAGE_SIZE,
    })
```

chat/urls.py

```
from django.urls import path
from . import views

app_name = "chat"

urlpatterns = [
    path("", views.inbox, name="inbox"),
    path("start/<int:user_id>/", views.start_chat, name="start"),
    path("room/<int:room_id>/", views.room, name="room"),
    path("room/<int:room_id>/history/", views.history, name="history"),
]
```

socialnest/urls.py (add one line)

```
urlpatterns = [
    # ... existing routes ...
    path("chat/", include("chat.urls")),
]
```

Done when: /chat/ shows (an empty) inbox when logged in and redirects to login when logged out.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Write Django views and urls for my chat app (namespace 'chat'): inbox (list rooms of request.user with other user, last message, unread count), start_chat/<int:user_id> (get_or_create_direct, refuse chatting with yourself, redirect to the room), room/<int:room_id> (404 unless the user is a participant, pass the last 30 messages oldest-first, mark incoming messages read) and room/<id>/history returning JSON of older messages with ?before=<id>. All login_required. Include the line to add to the project urls.py.

TASK 8 Inbox template ~10 min

Goal: List conversations with unread badges.

templates/chat/inbox.html

```
{% extends "base.html" %}
{% load static %}
{% block content %}
<link rel="stylesheet" href="{% static 'chat/chat.css' %}">
<div class="chat-inbox">
  <h2>Messages</h2>
  {% for c in conversations %}
    <a class="chat-item" href="{% url 'chat:room' c.room.id %}">
      <div class="avatar">{{ c.other.username|slice:":1"|upper }}</div>
      <div class="chat-item-body">
        <strong>{{ c.other.username }}</strong>
        <span class="preview">
          {% if c.last %}{{ c.last.content|truncatechars:45 }}{% else %}No messages yet{% endif %}
        </span>
      </div>
      {% if c.unread %}<span class="badge">{{ c.unread }}</span>{% endif %}
    </a>
  {% empty %}
    <p class="muted">No conversations yet. Open someone's profile and press <b>Message</b>.</p>
  {% endfor %}
</div>
{% endblock %}
```

Done when: Conversations created in the admin show up here.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Create a Django template templates/chat/inbox.html extending base.html (block content). Loop over 'conversations' (dicts with room, other, last, unread) and render an avatar initial, the other user's username, a 45-char preview of the last message, and a red unread badge when unread > 0. Link each item to {% url 'chat:room' c.room.id %}. Add an empty state.

TASK 9 Chat room template ~15 min

Goal: Page skeleton with message list, typing line, and input box.

templates/chat/room.html

```
{% extends "base.html" %}
{% load static %}
{% block content %}
<link rel="stylesheet" href="{% static 'chat/chat.css' %}">

<div class="chat-box">
  <header class="chat-header">
    <a href="{% url 'chat:inbox' %}">&larr;</a>
    <div class="avatar">{{ other.username|slice:":1"|upper }}</div>
    <div>
      <strong>{{ other.username }}</strong>
      <small id="chat-status" class="muted">Connecting...</small>
    </div>
  </header>

  <div id="chat-log" class="chat-log">
    {% for m in chat_messages %}
      <div class="msg {% if m.sender_id == request.user.id %}mine{% else %}theirs{% endif %}"
           data-id="{{ m.id }}">
        <div class="bubble">{{ m.content }}</div>
        <time>{{ m.timestamp|date:"H:i" }}</time>
      </div>
    {% endfor %}
  </div>

  <div id="typing-line" class="typing" hidden>{{ other.username }} is typing...</div>

  <div class="chat-input">
    <input id="chat-text" type="text" maxlength="2000" placeholder="Type a message..." autocomplete="off">
    <button id="chat-send" type="button">Send</button>
```

```
  </div>
</div>

{{ room.id|json_script:"room-id" }}
{{ request.user.id|json_script:"me-id" }}
<script src="{% static 'chat/chat.js' %}"></script>
{% endblock %}
```

Why json_script? It safely passes Python values into JavaScript with correct escaping, with no inline string-building that could cause XSS.

Done when: Opening /chat/room/1/ shows the stored messages (live updates come in Task 10).

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

Create templates/chat/room.html extending base.html: header with back link, avatar and a status element id=chat-status; a scrollable div id=chat-log rendering existing 'chat_messages' as bubbles with class mine/theirs (compare m.sender_id with request.user.id) and data-id; a hidden typing line id=typing-line; an input id=chat-text and button id=chat-send. Expose room.id and request.user.id with json_script (ids room-id and me-id) and load static chat/chat.js.

TASK 10 Frontend JavaScript (WebSocket client) ~30 min

Goal: Connect, send, receive, show typing, mark read, and auto-reconnect.

static/chat/chat.js

```
(function () {
  const roomId = JSON.parse(document.getElementById("room-id").textContent);
  const meId = JSON.parse(document.getElementById("me-id").textContent);
  const log = document.getElementById("chat-log");
  const input = document.getElementById("chat-text");
  const sendBtn = document.getElementById("chat-send");
  const statusEl = document.getElementById("chat-status");
  const typingEl = document.getElementById("typing-line");

  const scheme = location.protocol === "https:" ? "wss" : "ws";
  const url = `${scheme}://${location.host}/ws/chat/${roomId}/`;

  let socket = null;
  let retries = 0;
  let typingTimer = null;
  let iAmTyping = false;

  function scrollDown() { log.scrollTop = log.scrollHeight; }

  function fmtTime(iso) {
    const d = new Date(iso);
    return d.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" });
  }

  // textContent (never innerHTML) so user text can never inject HTML
  function addMessage(m) {
    if (log.querySelector(`[data-id="${m.id}"]`)) return;          // avoid duplicates
    const wrap = document.createElement("div");
    wrap.className = "msg " + (m.sender_id === meId ? "mine" : "theirs");
    wrap.dataset.id = m.id;
    const bubble = document.createElement("div");
    bubble.className = "bubble";
    bubble.textContent = m.message;
    const time = document.createElement("time");
    time.textContent = fmtTime(m.timestamp);
    wrap.append(bubble, time);
    log.appendChild(wrap);
    scrollDown();
  }

  function send(obj) {
```

```
    if (socket && socket.readyState === WebSocket.OPEN) {
      socket.send(JSON.stringify(obj));
      return true;
    }
    return false;
  }

  function connect() {
    statusEl.textContent = retries ? "Reconnecting..." : "Connecting...";
    socket = new WebSocket(url);

    socket.onopen = function () {
      retries = 0;
      statusEl.textContent = "Online";
      send({ type: "read" });
    };

    socket.onmessage = function (e) {
      const d = JSON.parse(e.data);
      if (d.type === "message") {
        addMessage(d);
        if (d.sender_id !== meId) send({ type: "read" });
      } else if (d.type === "typing") {
        typingEl.hidden = !d.is_typing;
      }
    };

    socket.onclose = function (e) {
      if (e.code === 4403) { statusEl.textContent = "Not allowed in this chat"; return; }
      statusEl.textContent = "Disconnected. Retrying...";
      const wait = Math.min(1000 * 2 ** retries, 15000);          // 1s, 2s, 4s ... max 15s
      retries += 1;
      setTimeout(connect, wait);
    };

    socket.onerror = function () { socket.close(); };
  }

  function submit() {
    const text = input.value.trim();
    if (!text) return;
    if (send({ type: "message", message: text })) {
      input.value = "";
      stopTyping();
    } else {
      statusEl.textContent = "Not connected yet. Try again in a moment.";
    }
  }

  function stopTyping() {
    if (iAmTyping) { iAmTyping = false; send({ type: "typing", is_typing: false }); }
  }

  input.addEventListener("input", function () {
    if (!iAmTyping) { iAmTyping = true; send({ type: "typing", is_typing: true }); }
    clearTimeout(typingTimer);
    typingTimer = setTimeout(stopTyping, 1500);
  });
  input.addEventListener("keydown", function (e) {
    if (e.key === "Enter" && !e.shiftKey) { e.preventDefault(); submit(); }
  });
  sendBtn.addEventListener("click", submit);

  scrollDown();
  connect();
})();
```

Done when: Open the same room in two browsers (one normal, one private window, two different users). Messages appear instantly on both sides.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Write vanilla JavaScript (static/chat/chat.js) for a Django Channels chat room. Read room-id and me-id from json_script elements. Connect to ws(s)://host/ws/chat/<room>/ choosing wss on https. Send JSON {type:'message', message}, {type:'typing', is_typing}, {type:'read'}. Render incoming messages with textContent (no innerHTML), skip duplicates by data-id, auto-scroll, show/hide a typing line, update a status label, and reconnect with exponential backoff (1s doubling to 15s) except on close code 4403. Enter sends, Shift+Enter does not.

TASK 11 Styling ~15 min

Goal: Make it look like a modern messenger.

static/chat/chat.css

```
.chat-box, .chat-inbox { max-width: 640px; margin: 1.5rem auto; background: #fff;
  border: 1px solid #e2e8f0; border-radius: 14px; overflow: hidden; }
.chat-inbox { padding: 1rem; }
.chat-header { display: flex; align-items: center; gap: .75rem; padding: .75rem 1rem;
  border-bottom: 1px solid #e2e8f0; background: #f8fafc; }
.chat-header a { text-decoration: none; font-size: 1.3rem; color: #334155; }
.avatar { width: 38px; height: 38px; border-radius: 50%; background: #4f46e5; color: #fff;
  display: grid; place-items: center; font-weight: 700; flex: none; }
.muted { color: #64748b; display: block; font-size: .78rem; }

.chat-log { height: 62vh; overflow-y: auto; padding: 1rem; display: flex;
  flex-direction: column; gap: .5rem; background: #f1f5f9; }
.msg { max-width: 75%; display: flex; flex-direction: column; }
.msg.mine { align-self: flex-end; align-items: flex-end; }
.msg.theirs { align-self: flex-start; }
.bubble { padding: .5rem .8rem; border-radius: 16px; word-wrap: break-word; white-space: pre-wrap; }
.mine .bubble { background: #4f46e5; color: #fff; border-bottom-right-radius: 4px; }
.theirs .bubble { background: #fff; color: #0f172a; border-bottom-left-radius: 4px; }
.msg time { font-size: .68rem; color: #94a3b8; margin-top: 2px; }

.typing { padding: .2rem 1rem; font-size: .8rem; color: #64748b; font-style: italic; }
.chat-input { display: flex; gap: .5rem; padding: .75rem; border-top: 1px solid #e2e8f0; }
.chat-input input { flex: 1; padding: .6rem .9rem; border: 1px solid #cbd5e1; border-radius: 999px; }
.chat-input button { padding: .6rem 1.2rem; border: 0; border-radius: 999px;
  background: #4f46e5; color: #fff; font-weight: 600; cursor: pointer; }

.chat-item { display: flex; align-items: center; gap: .75rem; padding: .7rem; border-radius: 10px;
  text-decoration: none; color: inherit; }
.chat-item:hover { background: #f1f5f9; }
.chat-item-body { flex: 1; display: flex; flex-direction: column; }
.preview { color: #64748b; font-size: .85rem; }
.badge { background: #ef4444; color: #fff; border-radius: 999px; padding: 0 .5rem;
  font-size: .75rem; font-weight: 700; }
```

Done when: Bubbles align left and right; the layout works on a phone-width window.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Write chat.css for a messenger UI: centered card (max 640px), header with avatar, scrollable log (flex column), my bubbles right and indigo, their bubbles left and white, small timestamps, typing line, pill input with send button, inbox rows with hover and a red badge. Keep it framework-free.

TASK 12 Connect chat to the rest of SocialNest ~15 min

Goal: Add the Message button on profiles and a Messages link with unread count in the navbar.

templates/accounts/profile.html (next to Follow button)

```
{% if request.user.is_authenticated and profile_user != request.user %}
  <a class="btn" href="{% url 'chat:start' profile_user.id %}">Message</a>
{% endif %}
```

Replace `profile_user` with the variable your profile view passes (it may be `user`, `profile.user`, etc.).

chat/context_processors.py

```
from .models import Message


def unread_count(request):
    if request.user.is_authenticated:
        n = (Message.objects.filter(room__participants=request.user, is_read=False)
             .exclude(sender=request.user).count())
        return {"chat_unread": n}
    return {"chat_unread": 0}
```

socialnest/settings.py (TEMPLATES -> OPTIONS -> context_processors)

```
"context_processors": [
    # ... existing ...
    "chat.context_processors.unread_count",
],
```

templates/base.html (inside your navbar)

```
<a href="{% url 'chat:inbox' %}">Messages
  {% if chat_unread %}<span class="badge">{{ chat_unread }}</span>{% endif %}
</a>
```

Done when: Click Message on a profile, land in a room, and the navbar badge shows unread counts for the other user.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Here are my profile.html and base.html: <paste both>. Add a Message button linking to {% url 'chat:start' <profile user id> %} shown only to logged-in users viewing someone else's profile, and a Messages navbar link with an unread badge from a context variable chat_unread. Also write chat/context_processors.py computing chat_unread and show where to register it in settings.TEMPLATES.

TASK 13 Run and manually test ~15 min

Goal: Verify the real thing with two users.

terminal

```
python manage.py createsuperuser          # or register two normal users in the app
python manage.py runserver
```

- Browser 1: log in as **alice**. Browser 2 (private window): log in as **bob**.
- Alice opens Bob's profile and presses **Message**. Bob opens `/chat/` and the same room.
- Type in Alice's window: Bob sees 'alice is typing...' then the message, with no page refresh.
- Refresh both pages: history is still there (it came from the database).
- Log in as a third user and open `/chat/room/<that id>/`: you get 404. Try the WebSocket URL directly: it is rejected (code 4403).
- Stop the server for 5 seconds, restart: the status text switches to 'Retrying' then 'Online'.

Done when: Every bullet above behaves as described.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> My Django Channels chat is not working. Symptoms: <describe>. Browser console says: <paste>. Server log says: <paste>. Here are my settings.py, asgi.py, routing.py, consumers.py: <paste>. Walk through the connection path (asgi -> middleware -> routing -> consumer -> group) and tell me the most likely cause first.

TASK 14 Automated tests ~20 min

Goal: Lock in security and delivery behaviour.

chat/tests.py

```
from channels.routing import URLRouter
from channels.testing import WebsocketCommunicator
from django.contrib.auth import get_user_model
from django.test import TestCase, TransactionTestCase

from .models import ChatRoom, Message
from .routing import websocket_urlpatterns

User = get_user_model()


class ModelTests(TestCase):
    def test_direct_room_is_reused(self):
        a = User.objects.create_user("a", password="x")
        b = User.objects.create_user("b", password="x")
        r1, created1 = ChatRoom.get_or_create_direct(a, b)
        r2, created2 = ChatRoom.get_or_create_direct(b, a)
        self.assertTrue(created1)
        self.assertFalse(created2)
        self.assertEqual(r1.id, r2.id)


class ConsumerTests(TransactionTestCase):
    def setUp(self):
        self.a = User.objects.create_user("alice", password="x")
        self.b = User.objects.create_user("bob", password="x")
        self.c = User.objects.create_user("carol", password="x")
        self.room, _ = ChatRoom.get_or_create_direct(self.a, self.b)
        self.app = URLRouter(websocket_urlpatterns)

    async def _connect(self, user):
        comm = WebsocketCommunicator(self.app, f"/ws/chat/{self.room.id}/")
        comm.scope["user"] = user
        ok, _ = await comm.connect()
        return comm, ok

    async def test_member_sends_and_receives(self):
        ca, ok_a = await self._connect(self.a)
        cb, ok_b = await self._connect(self.b)
        self.assertTrue(ok_a and ok_b)
        await ca.send_json_to({"type": "message", "message": "hello"})
        got = await cb.receive_json_from()
        self.assertEqual(got["message"], "hello")
        self.assertEqual(got["sender"], "alice")
        await ca.disconnect()
        await cb.disconnect()

    async def test_non_member_rejected(self):
        _, ok = await self._connect(self.c)
        self.assertFalse(ok)

    async def test_empty_message_ignored(self):
        ca, _ = await self._connect(self.a)
        await ca.send_json_to({"type": "message", "message": "   "})
        self.assertTrue(await ca.receive_nothing())
        await ca.disconnect()
```

terminal

```
python manage.py test chat
```

Done when: All tests pass.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> Write Django tests for my chat app using channels.testing.WebsocketCommunicator inside a TransactionTestCase: member can send and the other member receives it, non-member connection is rejected, whitespace-only message is ignored, plus a TestCase proving get_or_create_direct reuses the room regardless of argument order. Models: ChatRoom, Message. Routing: chat.routing.websocket_urlpatterns.

TASK 15 Production: Redis, Daphne, and a WebSocket-aware proxy ~30 min

Goal: Move from the dev setup to something that survives real traffic.

15a. Redis channel layer

InMemoryChannelLayer only works inside one process. With more than one worker (or any restart) messages will not reach other users, so use Redis.

terminal

```
# Ubuntu server
sudo apt install redis-server
sudo systemctl enable --now redis-server
```

socialnest/settings.py

```
import os

CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {"hosts": [os.environ.get("REDIS_URL", "redis://127.0.0.1:6379")]},
    },
}
ALLOWED_HOSTS = ["yourdomain.com"]
CSRF_TRUSTED_ORIGINS = ["https://yourdomain.com"]
```

15b. Run with Daphne (or Uvicorn)

terminal

```
daphne -b 127.0.0.1 -p 8000 socialnest.asgi:application
```

15c. Nginx must forward the WebSocket upgrade

/etc/nginx/sites-available/socialnest

```
location /ws/ {
    proxy_pass http://127.0.0.1:8000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_read_timeout 86400;
}
location / {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

- Serve the site over HTTPS so the browser uses `wss://` (the JS already switches automatically).
- Run Daphne under systemd or supervisor so it restarts on failure.
- Platforms like Render, Railway or Fly.io support WebSockets; classic shared hosting and PythonAnywhere free tier do not.
- Serve static files with WhiteNoise or Nginx (`collectstatic`).

Done when: Two users on different devices chat through your public HTTPS domain.

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

I want to deploy my Django Channels chat app on <Ubuntu VPS / Render / Railway>. Give me: the production CHANNEL_LAYERS config using Redis from an environment variable, the exact command to run Daphne, a systemd unit file, an Nginx server block that proxies /ws/ with the Upgrade headers, and the Django settings (ALLOWED_HOSTS, CSRF_TRUSTED_ORIGINS, SECURE_PROXY_SSL_HEADER) needed behind HTTPS.

Part 5. Optional Upgrades

Feature	How
Online / last seen	Group 'presence'; on connect/disconnect update a UserPresence row or Redis key; broadcast to followers.
Group chats	Create ChatRoom(is_group=True), add many participants. The consumer already works unchanged.
Image / file sharing	Upload via normal HTTP POST (FileField on a MessageAttachment), then broadcast the file URL over the socket.
Message delete / edit	HTTP endpoint changes the row, then group_send a 'chat.deleted' / 'chat.edited' event.
Browser notifications	Notification API when a message arrives while the tab is hidden; or a global notifications consumer per user.
Rate limiting	Count messages per user per minute in Redis inside receive(); close or ignore when exceeded.
REST API / mobile app	Add Django REST Framework endpoints for rooms and history; pass a token in the WebSocket query string and validate it in custom middleware.

Part 6. Troubleshooting

Symptom	Likely cause and fix
WebSocket connection to ws://... failed (404 in log)	routing.py regex does not match the URL, or asgi.py is not the ASGI_APPLICATION. Check the pattern ^ws/chat/(?P<room_id>\d+)/$.
runserver says 'WSGI development server'	'daphne' is not the first item in INSTALLED_APPS, or Daphne is not installed.
Connects then closes immediately (code 4403)	User not logged in (session cookie missing) or not a participant of that room.
Connection closes with 403 right away	AllowedHostsOriginValidator: the host you used is missing from ALLOWED_HOSTS (e.g. 127.0.0.1 vs localhost).
AppRegistryNotReady in asgi.py	chat.routing imported before get_asgi_application() ran. Keep the import order from Task 6.
SynchronousOnlyOperation	ORM used directly inside async code. Wrap it in database_sync_to_async.
Messages arrive only for the sender	Handler method name wrong (type 'chat.message' needs method chat_message), or group names differ.
Works locally, not in production	No Redis layer, or Nginx missing Upgrade/Connection headers, or proxy_read_timeout too small.
Duplicate messages after reconnect	Keep the data-id duplicate check in addMessage().
Static files 404 in production	Run collectstatic and serve /static/ (WhiteNoise or Nginx).

Part 7. One-Shot Master Prompt

If you would rather let an AI coding assistant do the whole job, open your project in it and paste this. It works best in an agent that can read your repository (Claude Code, Cursor, etc.).

AI PROMPT (copy and paste into Claude Code / Cursor / ChatGPT):

> You are working inside my Django project (clone of FarhanAli995/codeAlpha_SocialNest). First read the repository: find the project package name, installed apps, base template and block names, how profiles are rendered, and which User model is used. Then add real-time 1-to-1 chat with Django Channels 4 + Daphne.

Deliver in this order and commit after each step: (1) requirements + settings (daphne first, channels, ASGI_APPLICATION, InMemory layer with Redis via env var for production); (2) new app 'chat' with models ChatRoom(name, is_group, participants M2M, created_at, updated_at) and Message(room, sender, content, timestamp, is_read) + get_or_create_direct; migrations + admin; (3) AsyncWebsocketConsumer with auth and membership check (close 4403), message/typing/read events, DB persistence via database_sync_to_async; (4) routing.py and asgi.py using ProtocolTypeRouter, AllowedHostsOriginValidator, AuthMiddlewareStack; (5) views (inbox, start_chat, room, history JSON) + urls under /chat/; (6) templates chat/inbox.html and chat/room.html extending my base template, static chat.js (auto-reconnect, textContent only, dedupe) and chat.css; (7) Message button on profile pages and a navbar Messages link with unread badge via a context processor; (8) tests with WebsocketCommunicator.
Rules: match my existing code style and template structure, do not break existing features, never use innerHTML for message text, and finish with the exact commands to migrate, test, and run. List any assumption you had to make.

Quick command cheat sheet

```
pip install "channels[daphne]" channels-redis
python manage.py startapp chat
python manage.py makemigrations chat && python manage.py migrate
python manage.py runserver                      # Daphne via INSTALLED_APPS
python manage.py test chat
daphne -b 127.0.0.1 -p 8000 socialnest.asgi:application    # production
```